WEEK – 9

TASK 1 : To Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 3

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main() {
    Job jobs[MAX_JOBS];
    int job_count = 0;

    /* Create background jobs */
    for (int i = 0; i < MAX_JOBS; i++) {
        pid_t pid = fork();

        if (pid == -1) {
            perror("fork");
            return 1;
        }

        if (pid == 0) {
            sleep(8 + i);
            exit(0);
        }
```

```
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;
        snprintf(jobs[job_count].state,
                 sizeof(jobs[job_count].state),
                 "Running");

        job_count++;
    }

    sleep(1);

    printf("\n%-8s %-10s %-15s\n", "Job ID", "PID", "Status");
    printf("--------------------------------\n");

    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

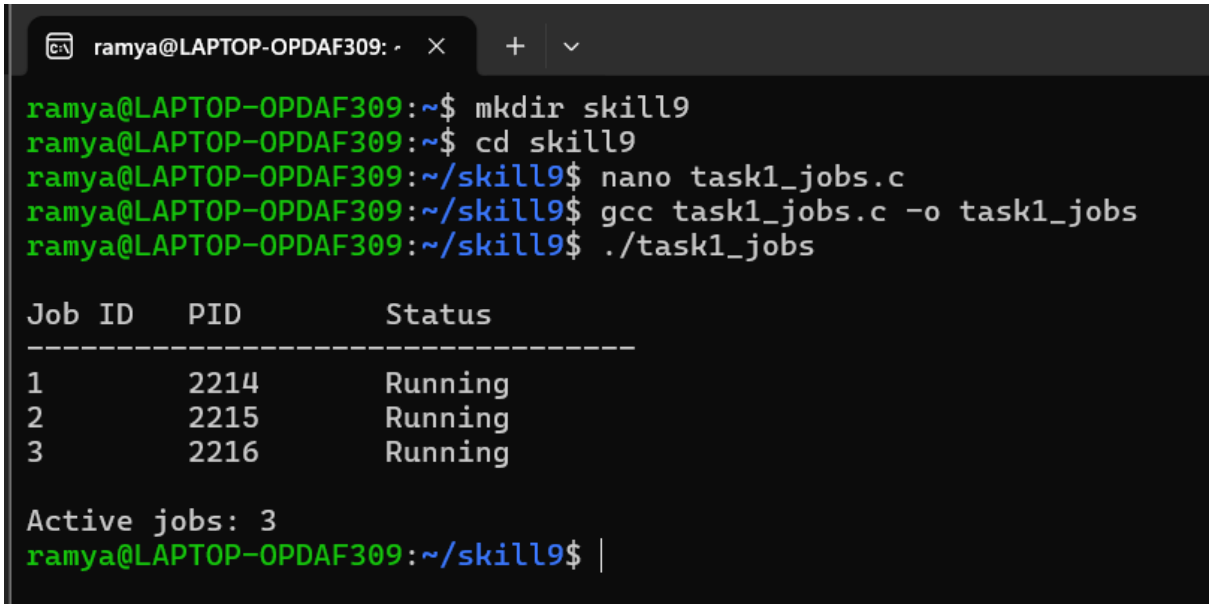
        if (result == jobs[i].pid) {
            snprintf(jobs[i].state,
                     sizeof(jobs[i].state),
                     "Completed");
        }

        printf("%-8d %-10d %-15s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].state);
    }
```

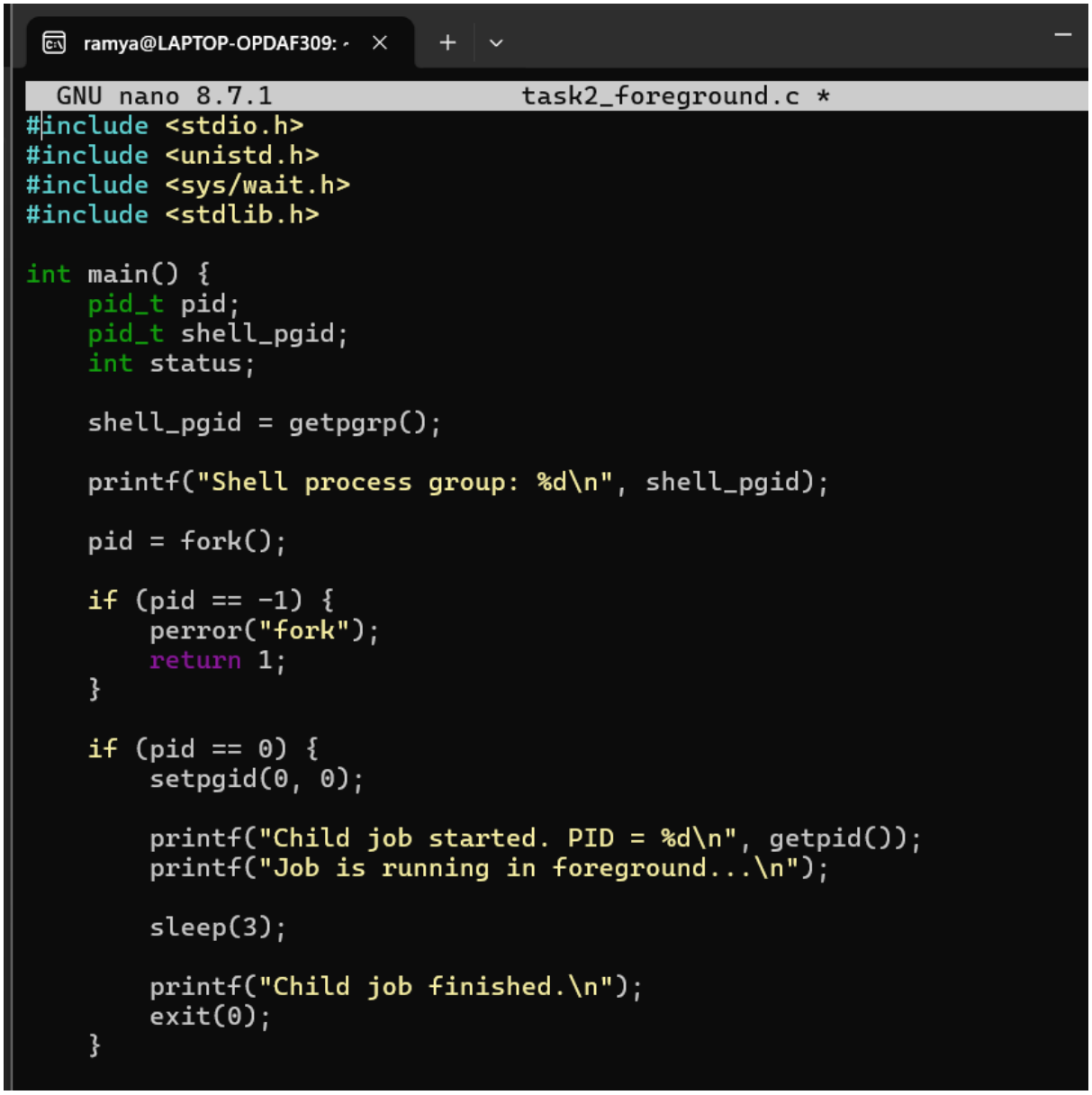
```
    printf("\nActive jobs: %d\n", job_count);

    /* Clean up child processes */
    for (int i = 0; i < job_count; i++)
        waitpid(jobs[i].pid, NULL, 0);

    return 0;
}
```



TASK 2 : To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task2_foreground.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
pid_t pid;
pid_t shell_pgid;
int status;
shell_pgid = getpgrp();
printf("Shell process group: %d\n", shell_pgid);
pid = fork();
if (pid == -1) {
perror("fork");
return 1;
}
if (pid == 0) {
setpgid(0, 0);
printf("Child job started. PID = %d\n", getpid());
printf("Job is running in foreground...\n");
sleep(3);
printf("Child job finished.\n");
exit(0);
}

```c
    setpgid(pid, pid);

    printf("Job PID: %d\n", pid);
    printf("Giving terminal control to the job...\n");

    if (tcsetpgrp(STDIN_FILENO, pid) == -1) {
        perror("tcsetpgrp");
    }

    waitpid(pid, &status, 0);

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
        perror("tcsetpgrp");
    }

    printf("Terminal control returned to parent.\n");

    if (WIFEXITED(status))
        printf("Foreground job completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill9$ gcc task2_foreground.c -o task2_foreground
ramya@LAPTOP-OPDAF309:~/skill9$ ./task2_foreground
Shell process group: 2371
Job PID: 2372
Giving terminal control to the job...
Child job started. PID = 2372
Job is running in foreground...
Child job finished.

[1]+  Stopped                 ./task2_foreground
```

TASK 3 : To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

void handle_sigint(int sig)
{
    printf("\nSIGINT received by process %d\n", getpid());
}

int main()
{
    pid_t pid;
    sigset_t mask;

    signal(SIGINT, handle_sigint);

    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return 1;
    }
```

```c
    if (pid == 0)
    {
        setpgid(0, 0);

        printf("Child PID: %d\n", getpid());
        printf("Child process group: %d\n", getpgrp());

        printf("\nBlocking SIGINT for 5 seconds...\n");

        sigprocmask(SIG_BLOCK, &mask, NULL);

        sleep(5);

        printf("Unblocking SIGINT...\n");

        sigprocmask(SIG_UNBLOCK, &mask, NULL);

        sleep(2);

        printf("Child finished.\n");

        exit(0);
    }

    sleep(1);

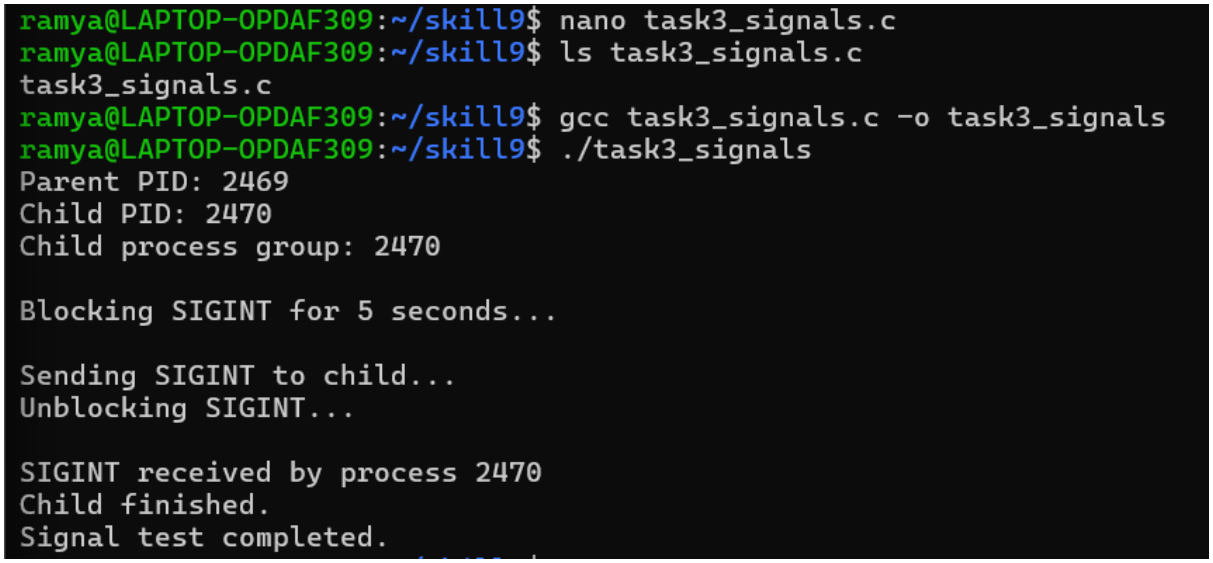
    printf("\nSending SIGINT to child...\n");

    kill(pid, SIGINT);

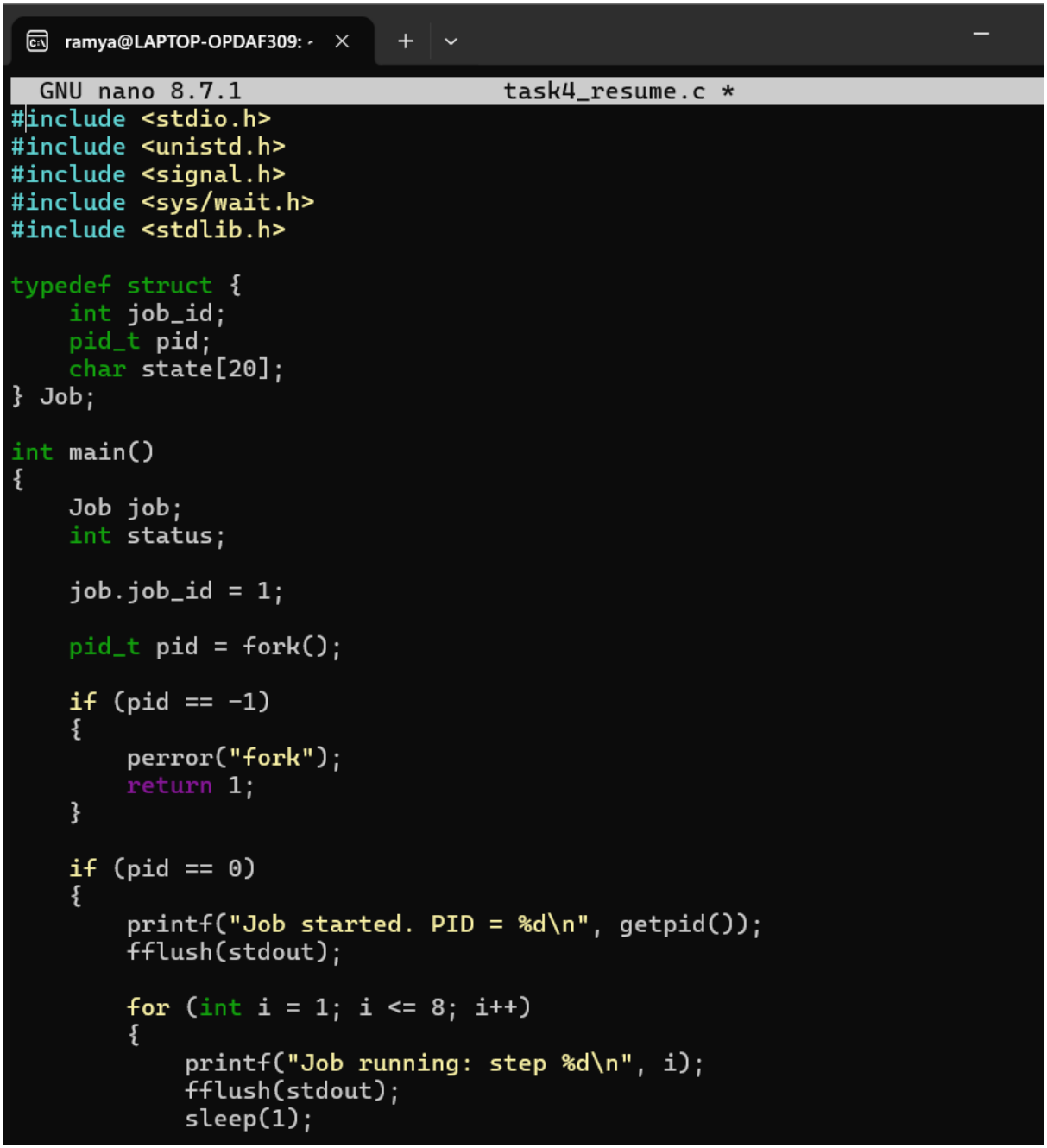
    waitpid(pid, NULL, 0);

    printf("Signal test completed.\n");

    return 0;
}
```



TASK 4 : To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task4_resume.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>
typedef struct {
int job_id;
pid_t pid;
char state[20];
} Job;
int main()
{
Job job;
int status;
job.job_id = 1;
pid_t pid = fork();
if (pid == -1)
{
perror("fork");
return 1;
}
if (pid == 0)
{
printf("Job started. PID = %d\n", getpid());
fflush(stdout);
for (int i = 1; i <= 8; i++)
{
printf("Job running: step %d\n", i);
fflush(stdout);
sleep(1);

```
        }

        printf("Job completed.\n");
        fflush(stdout);

        exit(0);
    }

    job.pid = pid;

    snprintf(job.state, sizeof(job.state), "Running");

    sleep(2);

    printf("\nStopping job %d...\n", job.job_id);

    kill(job.pid, SIGSTOP);

    waitpid(job.pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        snprintf(job.state, sizeof(job.state), "Stopped");
    }

    printf("Job ID: %d | PID: %d | State: %s\n",
           job.job_id, job.pid, job.state);

    sleep(2);

    printf("\nSending SIGCONT to resume job...\n");

    kill(job.pid, SIGCONT);

    snprintf(job.state, sizeof(job.state), "Running");

    printf("Job ID: %d | PID: %d | State: %s\n",
           job.job_id, job.pid, job.state);

    waitpid(job.pid, &status, 0);

    if (WIFEXITED(status))
    {
        snprintf(job.state, sizeof(job.state), "Completed");
    }

    printf("\nFinal job status:\n");

    printf("Job ID: %d | PID: %d | State: %s\n",
           job.job_id, job.pid, job.state);

    printf("\nBackground recovery completed successfully.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill9$ nano task4_resume.c
ramya@LAPTOP-OPDAF309:~/skill9$ ls task4_resume.c
task4_resume.c
ramya@LAPTOP-OPDAF309:~/skill9$ gcc task4_resume.c -o task4_resume
ramya@LAPTOP-OPDAF309:~/skill9$ ./task4_resume
Job started. PID = 2526
Job running: step 1
Job running: step 2

Stopping job 1...
Job ID: 1 | PID: 2526 | State: Stopped

Sending SIGCONT to resume job...
Job ID: 1 | PID: 2526 | State: Running
Job running: step 3
Job running: step 4
Job running: step 5
Job running: step 6
Job running: step 7
Job running: step 8
Job completed.

Final job status:
Job ID: 1 | PID: 2526 | State: Completed

Background recovery completed successfully.
```